BudgetMesh: Rate-Aware Training and Compression for Opaque Differentiable Meshes

Anonymous CVPR submission

Abstract

Differentiable opaque meshes, as introduced by MeshSplatting, render in any engine, but a trained scene occupies hundreds of megabytes and no compression method exists for them. We show that they cannot be compressed the way ordinary meshes or Gaussian splats are: because every pixel is covered by a single opaque triangle, simplifying a trained mesh loses 1.9 dB on average against training a mesh of that size from the start. The size of the file therefore has to be decided while the mesh is trained, and the one existing handle, a cap on the number of vertices, fixes neither the vertex count that survives training (0.68 to 1.06 of the cap) nor the bytes each vertex costs. We introduce BudgetMesh, a complete compression scheme spanning training, encoding and decoding. During training it runs the real encoder on the current mesh, converts a user-specified file size into a vertex count, grows the mesh toward that count by splitting the triangles with the largest rendering error, and trims the excess by edge collapse while optimization continues; a dedicated codec with a standalone decoder then writes one compact, engine-ready file. In one training run and without size statistics from other scenes, the file lands within 10% of the requested size in 83 of 89 runs on four datasets. At 20 to 5 MB the files are 31–122× smaller than the trained model (7–27× smaller than its quantized, zstd-compressed PLY) at 0.3–1.3 dB lower PSNR and 0.09–0.19 higher LPIPS.

1. Introduction

Differentiable rendering can now optimize an *opaque triangle mesh* directly from posed images [18, 20]: a set of triangles with per-vertex position, color and view-dependent appearance, drawn by an ordinary rasterizer. Unlike Gaussian splats [12], such a mesh renders in any engine and on any phone, with depth and collisions for free. Its weakness is size. A trained MeshSplatting [18] scene occupies 430–780 MB in 32-bit floats, while compressed Gaussian scenes reach 5–20 MB [3, 15]. No compression method exists for this representation, and the obvious ones fail.

The first obvious route is to train the mesh and then simplify it, as one does with any mesh. For opaque meshes this does not work: every pixel is covered by exactly one triangle, so each vertex is placed and colored for the neighbours it has, and a decimator leaves the survivors where they were. At the same vertex count, a mesh trained to that size from the start is better than a trained-then-simplified mesh on 62 of 63 scene–budget pairs, by 1.85 dB on average (Sec. 4.1). The second route is a standard mesh codec. Draco [7] and MPEG TFAN [17] assume clean manifold meshes with a few attributes per vertex; a MeshSplatting mesh is a stitched soup with 45 appearance coefficients per vertex, and with the appearance kept as attributes their files are 3–18× larger than ours, still 2.7–3.5× larger when Draco is handed our own quantized payload (Sec. 4.4). Neither route touches the number of vertices without the loss above, so the size of the file has to be decided *while the mesh is trained*.

MeshSplatting offers one handle for that, a cap on the number of vertices, and a cap is not a file size. After stitching and cleaning, the trained mesh keeps between 0.68 and 1.06 of the cap, and not smoothly: on one scene, two caps 3% apart end 14% apart in vertices, the smaller cap with the larger mesh. Each vertex then costs 8 to 13 bytes depending on the scene and the operating point. Turning megabytes into a cap therefore requires size statistics from earlier runs on similar scenes; such statistics do not carry over to another kind of scene, and a second training with a rescaled cap does not reliably repair a miss (Sec. 4.2).

We present BudgetMesh, which takes a target file size in megabytes and returns a file of that size after one training run, without statistics from other scenes. The idea is to put the encoder inside the training loop. At regular intervals BudgetMesh encodes the current mesh with the final codec and reads off what a vertex and a face cost in this scene; from these costs and the target it computes the vertex count the file can hold, and plans the remaining growth steps toward that count. Which triangles are split is decided by rendering error: each step splits the planned number of triangles with the largest error, in place of MeshSplatting's importance sampling. After growth, the measured size is compared with the target and the excess is removed by edge collapse, lowest error per freed byte first, while optimization continues, so the surviving vertices adapt, which simplification after training cannot offer. A codec designed for

this representation then writes one file, and a standalone decoder turns it into a standard indexed mesh.

On Mip-NeRF 360, Tanks and Temples, Deep Blending and NeRF-Synthetic, 83 of 89 files land within $\pm 10\%$ of the requested size, at a quality cost below 0.1 dB where it can be measured directly against the same growth rule run without a size target. At 20, 10 and 5 MB the files are $31\times$, $60\times$ and $122\times$ smaller than the trained model at -0.30, -0.66 and -1.32 dB. Our contributions are:

- We show that opaque differentiable meshes must be sized during training: simplification after training loses 1.85 dB on average at the same vertex count, on 62 of 63 scene–budget pairs across three datasets.
- We propose BudgetMesh, the first method that takes the file size of such a mesh as an input. It measures the encoded size during training, plans growth toward the vertex count the target can hold, splits by rendering error and trims by edge collapse with training still running. It needs one run and no size statistics from other scenes, and at the same vertex count its meshes are 0.2–0.3 dB better than MeshSplatting's.
- We introduce a codec for this representation, with implicit vertex order, edge-cache connectivity coding, neighbour-predicted positions and a two-level appearance dictionary with neighbour-context indices; it is 2.7–3.5$\times$ smaller than Draco carrying the same payload.

2. Related Work

Meshes from posed images. MobileNeRF [4], BakedSDF [27] and Binary Opacity Grids [21] extract a mesh from a trained neural field and bake appearance into textures, at 100–900 MB per scene. Meshes extracted from Gaussian scenes [8, 28, 29] reach 1 GB and more. MeshSplatting [18] and MiLo [20] optimize the mesh itself. None of these controls the file size; MeshSplatting's only small variant is an RGB-only file that drops view dependence.

Compressing Gaussian scenes. Pruning, vector quantization and learned entropy models reduce 3DGS [12] from 700 MB to 5–50 MB [2, 3, 5, 14–16, 25]. Two properties make this easier than our setting: Gaussians are independent primitives, so a trained scene can be pruned cheaply, and the rate is set by a hyperparameter swept per scene. Opaque meshes have neither property.

Mesh compression and simplification. Edgebreaker [22], shipped in Draco [7], and MPEG TFAN [17] code manifold connectivity at about 2 bits per triangle and predict attributes along the traversal. Edge collapse with quadric error [6, 10] is the standard way to reduce a mesh. Our collapse criterion, error per freed byte, follows the Lagrangian rate–distortion principle of video coding [23], and steering a size by measuring the encoder's output is how rate control works in video encoders. What is new is where both act: inside the optimization that creates the primitives, with the bytes measured by the final encoder on the mesh being trained.

3. Method

BudgetMesh keeps MeshSplatting's renderer, losses and schedule and changes how the mesh grows and shrinks so that the encoded file meets a target size T. The logic follows from the goal. To hit a size we must know what the mesh costs in bytes while it is still being trained (Sec. 3.2); the cost turns T into a vertex count and a growth plan (Sec. 3.3); whatever the plan could not foresee is removed by edge collapse during training (Sec. 3.4). Alg. 1 gives the loop and Sec. 3.5 the codec whose bytes are counted.

3.1. Preliminaries

MeshSplatting [18] represents a scene by V vertices, each with a position $\mathbf{x} \in \mathbb{R}^3$, a DC color $\mathbf{c} \in \mathbb{R}^3$ and 45 spherical-harmonic (SH) coefficients $\mathbf{h}$ of degrees 1–3, and F triangles over them. Training runs 30k iterations. Every 500 iterations until 15k it *densifies*: it samples triangles by an importance score and replaces each by four, inserting the three edge midpoints as new vertices; it also prunes triangles that render with low opacity. A vertex cap stops densification once V reaches the cap. At 11k a restricted Delaunay step stitches the soup into a connected mesh, an opacity floor then rises until every triangle is opaque, and a final cleaning removes triangles that cover no pixel. We keep all of this. Let R be the size of the encoded file in bytes. We write D_t for the *error of triangle* t: the per-pixel L_1 residual summed over the pixels whose front-most triangle is t (the rasterizer returns this id), averaged over the views rendered since the last decision and normalized per megapixel.

3.2. Measuring the file during training

Byte probe. Every 1000 iterations once MeshSplatting starts pruning (4k), a copy of the mesh is cleaned exactly as the final mesh will be and passed through the real encoder of Sec. 3.5. The file size is affine in the mesh,

$$R = b_v V + b_f F + c, \tag{1}$$

where c covers the dictionaries and the header. The probe returns R, the bytes per vertex b_v and per face b_f, c, and the fraction of vertices that survive cleaning; these replace the previous values. Measuring matters because the costs are not constants: b_v ranges from 8 to 13 bytes across scenes and operating points, and with fixed costs small files land 18% too large (Sec. 4.6).

Vertex count. With f faces per vertex, Eq. (1) gives the vertex count that the target buys:

$$V^\star = (\alpha T - c) \,/\, (b_v + f\, b_f). \tag{2}$$

During growth the mesh is an unstitched soup whose face-to-vertex ratio says little about the final mesh, so we fix $f = 2.5$; the factor α is discussed in Sec. 3.4.

3.3. Growing toward the vertex count

How many. With m growth steps left, the plan raises V geometrically, $V \to V\,(V^\star/V)^{1/m}$, which asks for

$$k = V\big((V^\star/V)^{1/m} - 1\big)\,/\,(3\sigma) \tag{3}$$

splits now: each split adds three vertices, of which a fraction σ survives MeshSplatting's pruning at the next step (σ is measured from the previous cohort). The plan is recomputed at every step with the latest probe, so an error in one step is spread over the steps that remain.

Which. MeshSplatting samples the triangles to split by an importance score that favours visible triangles. We split the k triangles with the largest error D_t: a triangle is refined because it is rendered wrongly. Written as a threshold, triangle t is split iff

$$D_t\,/\,\Delta R_{\text{split}} > \lambda, \qquad \Delta R_{\text{split}} = 3(b_v + b_f), \tag{4}$$

with λ set to the k-th largest ratio. A split costs the same bytes for every triangle, so Eq. (4) is a ranking by error and λ is a result of the plan, not a parameter; we keep the form because the same quantity, error per byte, also decides collapses (Sec. 3.4), where the bytes do differ. Because the errors are re-measured between steps, the number of triangles that actually passes differs from k; we divide k by the measured yield y (passed over asked) of the last steps before taking the rank. Running Eq. (4) with a fixed λ instead gives training without a size target, which serves as the reference in Sec. 4.2; one such λ lands at files $2.5\times$ apart across scenes.

3.4. Trimming by collapse

What growth cannot see. After the mesh is stitched at 11k, the set of visible faces keeps changing as the opacity floor rises, and the stitched mesh has its own faces per vertex (2.0 to 3.5 on our scenes), none of which exists before 11k. The size reached by growth alone is therefore off by a scene-dependent amount, and the remainder is corrected on the stitched mesh.

Collapse rule. Collapsing edge (u, v) merges u and v into one vertex and deletes the faces on the edge. It frees $\Delta R_{uv} = b_v\,[\,u, v \text{ survive cleaning}\,] + b_f\,n_{uv}$ bytes, with n_{uv} the deleted faces that would survive cleaning, and it risks the error D_{uv} of the faces around u and v. Edges are collapsed in increasing order of $D_{uv}/\Delta R_{uv}$, as an independent set (no two collapses share a vertex) with a fold-over check. The merged vertex is placed at the midpoint and takes the average of the two attribute vectors and of their optimizer moments, so training continues without a restart and the survivors keep being optimized.

Algorithm 1 BudgetMesh training for a target size T

Require: images, target T, aim α
initialize MeshSplatting; byte costs (b_v, b_f, c) from defaults
for iteration $i = 1 \ldots 30$k **do**
  MeshSplatting optimization step; accumulate D_t ▷ per-triangle error
  if growth step ($i \le 15$k) **then**
    probe every 1k: encode a cleaned copy $\to b_v, b_f, c$
    $V^\star \leftarrow (\alpha T - c)/(b_v + f b_f)$ ▷ Eq. 2
    $k \leftarrow$ geometric plan toward $V^\star$, survival σ ▷ Eq. 3
    split the k/y triangles with the largest D_t ▷ Eq. 4
  if trimming step ($12.5\text{k} \le i \le 19.5$k) **then**
    $\hat{R} \leftarrow$ probe, extrapolated to end of ramp
    if $\hat{R} > T$ **then**
      collapse lowest $D_{uv}/\Delta R_{uv}$ until $\hat{R} \le T$
final cleaning; encode (Sec. 3.5) $\to$ one file

When and how much. From 12.5k to 19.5k, every 500 iterations, the probe measures R and a least-squares fit of the probe history extrapolates it to the end of the opacity ramp. If the estimate exceeds T, edges are collapsed until the excess is removed, at most 15% of the vertices per round. If it does not, nothing is collapsed: trimming removes bytes and cannot add them.

Growth aim. Since trimming is one-sided, the plan aims above the target by the factor α of Eq. (2). α stands for one number per scene that growth cannot observe, the faces per vertex of the final mesh: over 18 runs without trimming, the aim that would have landed exactly on the target correlates with it at $r = -0.97$, and no quantity available during growth predicts it on both captured scenes and objects (supplement). We therefore use one value per capture class, 1.25 for captured scenes and 1.1 for synthetic objects, chosen on Mip-NeRF 360 and NeRF-Synthetic, and let trimming absorb the per-scene remainder. The other settings are fixed for all runs: $f = 2.5$ during growth, a probe every 1000 iterations, trimming rounds as above; the initial byte costs are overwritten by the first probe.

3.5. Codec

The trained mesh stores 52 floats per vertex and three 32-bit indices per face. The codec makes three lossy reductions, then codes the result losslessly in an order where each stream is predicted from what the decoder already has.

Lossy reductions. (i) Positions are quantized on a per-axis uniform grid of $b = \lceil \log_2(\text{extent}/(0.1\,\bar{e})) \rceil$ bits, $\bar{e}$ the median edge length (14–18 bits on our scenes): the step is a tenth of a typical edge; one bit finer costs bytes for no PSNR, one bit coarser loses 0.06–0.2 dB. (ii) The 45 SH coefficients are 87% of the floats, and neighbouring vertices on a surface share their view-dependent behaviour, but the coefficients are too noisy to predict from one another. We therefore replace them by a two-level per-scene dictionary,

$\mathbf{h}_v \approx \mathbf{B}[i_v] + \mathbf{B}_r[j_v]$, with $|\mathbf{B}| = 4096$ and $|\mathbf{B}_r| = 256$ entries fitted by k-means and refined for 400 steps against the rendering loss with the assignments fixed. Each vertex keeps two indices. (iii) The DC color is quantized to 7 bits per channel, and the opacity, uniform after training, is one scalar.

Vertex order. Faces are reordered into a cache-friendly strip order [11], and vertex ids are assigned in order of first reference. The decoder can then infer every id: a vertex that has not appeared yet is always the next id. No permutation is transmitted, and every later stream can use the already-decoded neighbours of a vertex.

Connectivity. The coder keeps the 64 most recently decoded directed edges. A face that shares an edge with one of them is coded as *hit*: the cache slot, the edge direction and the third vertex; otherwise as *miss*: three vertices. A vertex is coded as 0 when it is new and as the difference to the last coded id otherwise. All symbols are coded with an adaptive binary arithmetic coder whose contexts are the symbol type and the bits already coded; no tables are sent. Edgebreaker-style coders traverse a manifold; our meshes are stitched soups with 45% boundary edges and thousands of components, which forces Edgebreaker to split vertices, so a local cache is the better model here. Connectivity costs about 15 bits per face and is the largest stream (39–43% of the file).

Positions. Each quantized position is predicted by its earliest already-decoded mesh neighbour, rebuilt by the decoder from the faces, and the residual is arithmetic coded per axis with the magnitude of the previous residual as context: about 16 bits per vertex against 45 raw.

Appearance indices. Index frequencies are nearly uniform, so an order-0 entropy coder gains nothing; the redundancy is spatial. For vertex v the coder lists the distinct indices of its already-decoded neighbours and codes one flag (is i_v among them) and, if so, its position in the list; otherwise the index itself. The DC colors are byte-shuffled and compressed with zstd.

Decoding. The decoder reads the connectivity, rebuilds the neighbour graph from it, then decodes positions and indices and expands $\mathbf{h}_v$ from the dictionaries. It needs nothing but the file, runs in 2–6 s on one CPU core, and outputs a standard indexed mesh that an ordinary vertex shader draws. The encoder verifies a byte-exact round trip before writing; all loss comes from the three reductions above.

4. Experiments

Datasets and protocol. Mip-NeRF 360 [1] (9 scenes), Tanks and Temples [13] (truck, train), NeRF-Synthetic [19] (8 objects) and Deep Blending [9] (drjohnson, playroom). Deep Blending was not used while developing the method; every number on it comes from one run with the settings fixed on the other datasets. MeshSplatting training is unchanged (30k iterations, images downsampled by 2 indoors and by 4 outdoors, the standard test split, black background for NeRF-Synthetic). Quality is PSNR, SSIM and LPIPS of the *decoded file* rendered at the renderer's supersampling; all sizes are bytes of the single delivered file. The *FP32 model* is the trained MeshSplatting mesh in 32-bit floats, our reproduction; it is the reference for compression ratios and quality loss. One configuration for all scenes: appearance dictionary $4096 + 256$, position grid of 14–18 bits by scene extent, 7-bit DC color, growth aim $\alpha = 1.25$ for captured scenes and 1.1 for synthetic objects, collapse rounds every 500 iterations from 12.5k to 19.5k. "At the same vertex count" means after MeshSplatting's final cleaning.

Table 1. **Deciding the size during training.** PSNR gain in dB, mean over pairs, with the number of pairs where the first method is better. (a) MeshSplatting trained with a vertex cap vs. the same FP32 model simplified to that vertex count after training. (b) BudgetMesh vs. MeshSplatting trained with a vertex cap chosen to end at BudgetMesh's vertex count: at the same vertex count before any entropy coding and after our codec, and at the same file size. Three counts / sizes per scene.

(a) trained to the count vs. simplified after training

	pairs	gain	range
Mip-360 + T&T	33	+2.07 (33)	+0.10 … +5.48
Deep Blending	6	+2.71 (6)	+1.39 … +4.72
NeRF-Synthetic	24	+1.33 (23)	−0.03 … +4.35

(b) BudgetMesh vs. MeshSplatting with a vertex cap

	same V, FP32	same V, coded	same size
Mip-360 + T&T	+0.27 (29/30)	+0.26 (29/30)	+0.21 (30/33)
NeRF-Synthetic	+0.24 (15/15)	+0.22 (15/15)	+0.12 (16/24)

Deep Blending in (b): same file size only, +0.08 (4/6); see text.

4.1. Size must be decided during training

Table 1 answers the two questions a reader will ask first. *Why not decimate a trained mesh?* For each scene and three vertex counts (those of BudgetMesh's own 20 / 10 / 5 MB files; 4 / 2.5 / 1.5 MB on NeRF-Synthetic) we compare MeshSplatting trained with that count as a vertex cap against the FP32 model simplified to the same count by edge collapse (quadric error with an appearance term) followed by the same 3000-step fine-tuning, both through our codec. Training to the count wins on 62 of 63 pairs, by 1.85 dB on average and by up to 5.5 dB; the gap grows as the budget shrinks (0.5 dB at the largest, 3.8 dB at the smallest count on real scenes). The simplified meshes keep fewer faces per vertex and show holes and speckle (supplement). *Is a mesh grown by BudgetMesh as good as one grown by MeshSplatting to the same count?* Against MeshSplatting trained with the cap set from BudgetMesh's own vertex count (pairs within $\pm 15\%$), BudgetMesh is better on 44 of 45 pairs, by 0.27 dB on real scenes and 0.24 dB on objects, measured on the FP32 meshes *before any entropy*

Table 2. **Size control and quality per dataset and target.** Mean file size; size error $(\text{file size} - T)/T$ as the mean and the range over scenes; scenes within $\pm 10\%$; PSNR of the decoded file and its change against the FP32 model; SSIM and LPIPS. Means over the scenes below their capacity limit; capacity-limited objects (the file cannot grow to T) are counted in parentheses. The 15, 7.5 and 3 MB targets are in the supplement.

dataset	target T	scenes	file size	size error	range	within $\pm 10\%$	PSNR (Δ)	SSIM	LPIPS
Mip-NeRF 360 + T&T	20 MB	11	19.9 MB	-0.5%	$-6.6 \dots +10.0\%$	10	23.75 (-0.30)	0.712	0.397
	10 MB	11	10.2 MB	$+2.0\%$	$-4.7 \dots +13.5\%$	10	23.39 (-0.66)	0.684	0.442
	5 MB	11	5.0 MB	$+0.6\%$	$-4.8 \dots +9.2\%$	11	22.72 (-1.32)	0.644	0.491
Deep Blending	20 MB	2	21.3 MB	$+6.7\%$	$+3.6 \dots +9.7\%$	2	26.94 (-0.28)	0.838	0.452
	10 MB	2	11.0 MB	$+10.2\%$	$+8.3 \dots +12.0\%$	1	26.54 (-0.69)	0.830	0.478
	5 MB	2	5.4 MB	$+7.9\%$	$+4.7 \dots +11.1\%$	1	25.89 (-1.34)	0.817	0.502
NeRF-Synthetic	4 MB	6 (+2)	3.9 MB	-2.9%	$-10.3 \dots +3.4\%$	5	30.60 (-0.13)	0.942	0.082
	2.5 MB	7 (+1)	2.6 MB	$+2.6\%$	$-6.6 \dots +9.2\%$	7	30.81 (-0.30)	0.947	0.076
	1.5 MB	8	1.6 MB	$+4.0\%$	$+1.0 \dots +9.0\%$	8	30.36 (-0.88)	0.944	0.080

coding; the margin is unchanged after the codec, so it comes from how the mesh is grown, not from the encoder. It is small next to the 1.85 dB above: where the size is decided matters far more than which triangles are split. At the same file size after the codec the margin is 0.21 dB on real scenes (30/33); on NeRF-Synthetic it is 0.12 dB (16/24) because at 4 MB two objects are at their capacity limit and a third is close to it (Sec. 4.2); at 1.5 MB it is 0.24 dB on 8 of 8. On Deep Blending the vertex-cap runs exist only at other vertex counts, so we report the same-file-size comparison alone: $+0.08$ dB, 4 of 6.

4.2. Size as an input

Table 2 reports, per dataset and target, the mean file size, the size error $(\text{file size} - \text{target})/\text{target}$ and the count of scenes within $\pm 10\%$; Fig. 1 draws the same runs as rate–distortion curves, with the per-scene spread of the landing as whiskers at each target. Across the four datasets 83 of 89 runs land within $\pm 10\%$ with one setting per capture class. The misses are systematic and we report them: indoor scenes land 5–14% above the target (kitchen at 10 MB is the worst, $+13.5\%$), because the visible-face set keeps growing after the growth phase and the collapse rounds can remove only so much per round; outdoor scenes at 20 MB

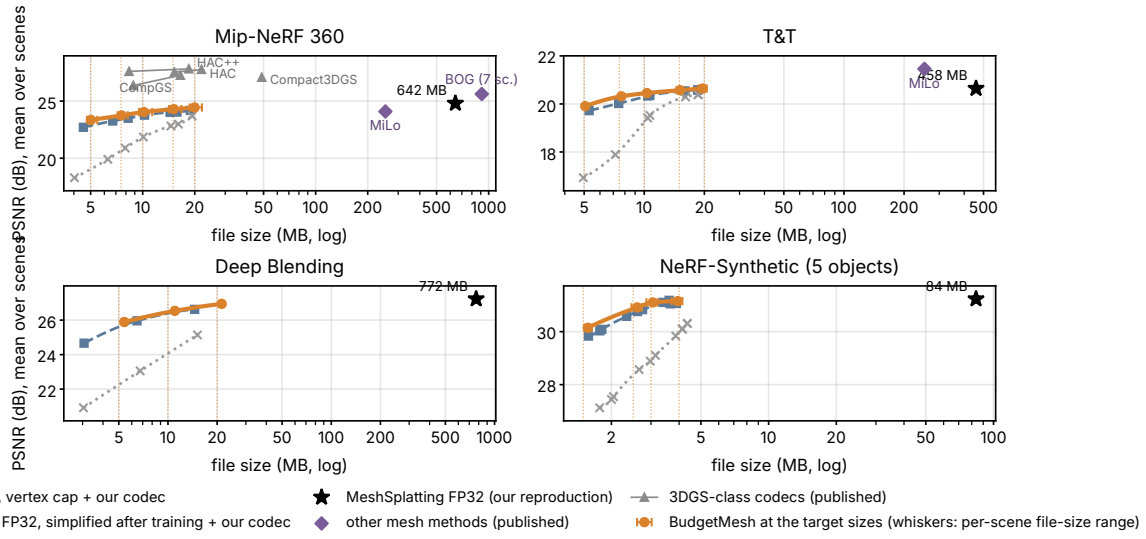


Figure 1. **Rate–distortion per dataset.** Mean PSNR vs. mean file size. Orange: BudgetMesh; dotted lines mark the targets, whiskers the per-scene range of the file size. Blue: MeshSplatting with a vertex cap, same codec. Grey crosses: the FP32 model simplified after training, same codec. Star: FP32 model. Diamonds and triangles: published mesh and 3DGS-class methods at their own sizes. NeRF-Synthetic: the five objects below their capacity limit.

Table 3. **Against Gaussian codecs and mesh-output methods**, dataset means; size in MB. Gaussian rows are the published HAC++ benchmark [3]. Mesh rows: (a) published numbers collected in [18], where MiLo, RaDe-GS and GOF report one asset size, for Mip-NeRF 360 (n/r: not reported); (b) MobileNeRF and NeRF2Mesh as published; MeshSplatting reports no NeRF-Synthetic results, so its FP32 rows are our reproduction. BudgetMesh: T = 20 / 10 / 5 MB on scenes, 4 / 2.5 / 1.5 MB on objects. ‡Black background; published NeRF-Synthetic rows use white. Deep Blending is in the supplement.

(a) Mip-NeRF 360 and Tanks and Temples

	Mip-NeRF 360				Tanks and Temples			
method	PSNR	SSIM	LPIPS	MB	PSNR	SSIM	LPIPS	MB
3DGS [12]	27.46	0.812	0.222	750.9	23.69	0.844	0.178	431.0
Scaffold-GS [16]	27.50	0.806	0.252	253.9	23.96	0.853	0.177	86.50
Compact3DGS [14]	27.08	0.798	0.247	48.80	23.32	0.831	0.201	39.43
LightGaussian [5]	27.00	0.799	0.249	44.54	22.83	0.822	0.242	22.43
HAC [2], low	27.53	0.807	0.238	15.26	24.04	0.846	0.187	8.10
HAC, high	27.77	0.811	0.230	21.87	24.40	0.853	0.177	11.24
ContextGS [26], low	27.62	0.808	0.237	12.68	24.20	0.852	0.184	7.05
ContextGS, high	27.75	0.811	0.231	18.41	24.29	0.855	0.176	11.80
HAC++ [3], low	27.60	0.803	0.253	8.34	24.22	0.849	0.190	5.18
HAC++, high	27.82	0.811	0.231	18.48	24.32	0.854	0.178	8.63
GOF mesh [28]	20.78	0.573	0.465	1500	21.69	0.690	0.326	n/r
RaDe-GS mesh [29]	23.56	0.668	0.361	1100	20.51	0.659	0.344	n/r
MiLo [20]	24.09	0.688	0.323	253	21.46	0.706	0.348	n/r
MeshSplatting FP32 [18]	24.81	0.731	0.309	642	20.64	0.760	0.272	459
BudgetMesh, T=20 MB	24.44	0.704	0.411	20.0	20.64	0.748	0.337	19.6
BudgetMesh, T=10 MB	24.04	0.673	0.458	10.2	20.45	0.731	0.368	10.3
BudgetMesh, T=5 MB	23.35	0.632	0.508	5.0	19.91	0.696	0.416	5.1

(b) NeRF-Synthetic‡

	8 objects			
method	PSNR	SSIM	LPIPS	MB
3DGS [12]	33.80	0.970	0.031	68.46
Scaffold-GS [16]	33.41	0.966	0.035	19.36
Compact3DGS [14]	33.33	0.968	0.034	5.54
LightGaussian [5]	32.73	0.965	0.037	7.84
HAC [2], low	33.24	0.967	0.037	1.18
HAC, high	33.71	0.968	0.034	1.86
ContextGS [26], low	32.79	0.965	0.040	1.01
ContextGS, high	33.51	0.968	0.035	1.56
HAC++ [3], low	33.03	0.966	0.039	0.88
HAC++, high	33.76	0.969	0.033	1.84
MobileNeRF [4]	30.90	0.947	0.062	125.8
NeRF2Mesh [24]	29.76	0.940	0.072	73.5
MeshSplatting FP32‡	31.24	[TBD:]	[TBD:]	65.7
BudgetMesh ‡, T=4 MB	31.13	0.951	0.070	3.49
BudgetMesh ‡, T=2.5 MB	30.97	0.950	0.072	2.49
BudgetMesh ‡, T=1.5 MB	30.36	0.944	0.080	1.56

Table 4. **Size control against cap-based alternatives**, measured on one scene per dataset: size error (%) of the delivered file for garden, truck, playroom (T = 10 MB) and lego (T = 2.5 MB), trainings used (runs), and scenes within ±10%. All rows use our codec.

method	runs	garden	truck	playr.	lego	≤10%
cap from prior[a]	1	+3.0	−0.5	+9.6	−41.8	3/4
cap rescaled[b]	2	+16.4	0.0	+16.3	−21.4	1/4
probe + plan only[c]	1	−1.7	−7.2	+12.0	+9.6	3/4
BudgetMesh	1	+2.0	+1.2	+8.3	−0.2	4/4

[a] File size fitted as a linear function of the cap on the cap runs of the other Mip-NeRF 360 / T&T scenes. [b] Cap multiplied by target over measured size, trained again. [c] Our probe and growth plan set the number of splits, MeshSplatting's importance sampling picks the triangles; no collapse.

land up to 7% below, because the last growth step admits fewer splits than planned and nothing can add vertices afterwards. On NeRF-Synthetic several objects cannot fill 3–6 MB at all: every triangle with a measurable error is split from the first steps and the file still stays small (ficus, mic; six of eight objects at 6 MB). These *capacity-limited* runs are counted separately and never hidden. Deep Blending, untouched during development, lands at +7 to +10% on average: the same indoor behaviour as Mip-NeRF 360, no retuning. A run takes 29–57 minutes on one L40S GPU including fine-tuning and encoding, of which the probes and collapse rounds are 10–16 minutes.

Could a vertex cap do the same? The natural alternative is MeshSplatting with a vertex cap chosen to give the wanted size (Table 4, one scene per dataset). The cap needs a size model; we give it the best one available, a linear fit of file size against cap on the cap runs of the other ten Mip-NeRF 360 / T&T scenes, files written by our codec. With this prior one training lands within 10% on the three captured scenes, including playroom from a dataset the prior never saw. On lego the same prior misses by 42%: bytes per vertex and the share of vertices that survive are different for objects, so the baseline needs statistics for every new kind of scene, which BudgetMesh measures on the scene itself. Retraining with the cap rescaled by the measured error does not converge: lego is still 21% off, and garden and playroom, retrained although they had landed, move from +3% and +10% to +16% with a *smaller* cap. Interpolating each scene's cap runs, we estimate that the prior-chosen cap would land within 10% on 9, 10 and 8 of the 11 Mip-NeRF 360 / T&T scenes at 20, 10 and 5 MB (an estimate, not runs), against 10, 10 and 11 measured for BudgetMesh. On captured scenes a good prior is thus competitive in hit rate; but it must exist, it does not transfer, and a miss cannot be repaired reliably.

Why a cap is a loose handle. The reason is visible in all 78 cap runs we have on captured scenes (same code, options and seed; only the cap differs). The vertices left after training are 0.68 to 1.06 of the cap (standard deviation 0.08),

and the ratio jumps inside one scene: stump keeps 0.75 of a cap of 888k and 0.92 of 983k, garden 0.80 of 1105k and 0.94 of 1073k. Reading every pair of caps of one scene that differ by at most 25% as a first and a second training, the second file is within 10% of what rescaling the first predicts in 48 of 62 pairs, and off by up to 22.5%. A cap acts before stitching, opacity pruning and cleaning, which then remove a share of the vertices that the cap does not control. BudgetMesh closes the loop after these stages, on the encoded bytes; three seeds of BudgetMesh on garden at 10 MB give 10.16, 10.16 and 10.21 MB. Replacing only the triangle selection by MeshSplatting's while keeping our probe and plan (third row) loses the collapse, which needs per-triangle errors, and lands within 10% on three of four; at its file size BudgetMesh is 0.17, 0.14 and 0.07 dB better on the captured scenes and 0.61 dB on lego.

Cost of control. The best a growth rule can do at a given file size is what it reaches when the size is *not* prescribed: the same rule run with a fixed threshold λ in Eq. (4), several trainings per scene, the size read off afterwards. Against that reference curve (three values of λ per scene, landing at about 5, 12 and 25 MB on real scenes and 1.5, 2.5 and 4 MB on objects), BudgetMesh is 0.04 dB lower at 20 MB, 0.06 dB at 2.5 MB and 0.02 dB at 1.5 MB, where a fixed-λ run lands at about the same size as the target, and 0.13 dB lower at 10 MB, where the curve is read between two runs (the curve is concave, so this chord reading under-estimates the cost; the three points bound the error at 0.19 dB on average over a span twice as wide). The cost of getting the size in one training is therefore below 0.1 dB where it can be measured directly and below about 0.3 dB everywhere. Run-to-run variance of the full method, three seeds on four scenes: the file size moves by 0.5–5% of the target and the PSNR by 0.01–0.14 dB. The reference also shows why a fixed threshold is not a size: at one λ the file size varies by 2.2–2.5× across the 11 real scenes (4.7–11.7 MB at one value, 10.8–24.0 MB at another).

4.3. Compression and other representations

Against the FP32 model the 20, 10 and 5 MB files are 31×, 60× and 122× smaller at −0.30, −0.66 and −1.32 dB on real scenes (Table 2); against the same meshes written as a quantized PLY and compressed with zstd they are 7×, 14× and 27× smaller. The ratio splits into two factors: the file holds fewer vertices (1.4×, 3.0× and 6.7× fewer than the 1.9–3.2M of the FP32 model), the growth rule's share, and each vertex costs 11–13 bytes instead of 231, the codec's share. Within the mesh class (Table 3a) BudgetMesh is the only entry below 250 MB, 0.4 dB below its own FP32 model at 20 MB and above MiLo, RaDe-GS and GOF at 13–75× smaller files; Binary Opacity Grids reports 25.62 dB at 913 MB on a 7-scene subset, without SSIM or LPIPS. Against compressed Gaussians (Table 3a), at 20 MB HAC++ and HAC are 3.3–4 dB higher in PSNR on Mip-NeRF 360 and Tanks and Temples (Deep Blending in the supplement), 0.10–0.11 higher in SSIM and 0.16–0.21 lower in LPIPS; on NeRF-Synthetic (Table 3b) the gap is 2.5–3 dB, with the caveat that our objects are rendered on black and the published ones on white. Within the mesh class on NeRF-Synthetic, BudgetMesh at 2.5 MB matches MobileNeRF's PSNR at 1/50 of its size and is 1.2 dB above NeRF2Mesh at 1/30 of its size. That gap is the representation's: opaque triangles against anisotropic Gaussians. What the mesh buys is a standard indexed mesh that renders at 145–350 fps on an iPhone 17 Pro at capture resolution and decodes in 2.3–6.4 s on one CPU core. The perceptual loss is larger than the PSNR loss suggests: on Mip-NeRF 360 LPIPS rises from 0.309 for the FP32 model to 0.411, 0.458 and 0.508 at 20, 10 and 5 MB, and SSIM falls from 0.731 to 0.704, 0.673 and 0.632; Sec. 4.5 traces this to the appearance dictionary.

Table 5. **Generic codecs on the same mesh.** File size of each route divided by ours, mean and range over the 11 Mip-NeRF 360 / T&T scenes, and the largest PSNR loss against FP32 (dB). The three "our payload" rows carry exactly our quantized positions, indices and DC, so their quality equals ours.

route	size ÷ ours	range	loss
Draco, 8-bit SH	15.0×	12.3–17.7	0.21
quantized PLY + zstd	5.2×	4.4–6.2	0.21
MPEG TFAN, 8-bit SH*	3.7×	3.0–4.2	0.21
Draco, our payload	3.0×	2.7–3.5	ours
Draco geometry + zstd payload	1.4×	1.3–1.6	ours
Draco geometry alone	1.05×	0.85–1.24	–
Draco, RGB only	2.1×	1.8–2.4	4.4
baked texture + Draco	2.7×	2.4–3.2	4.4
ours	1×	–	0.8

* 10 scenes. RGB-only routes drop view dependence.

4.4. Against generic mesh codecs

Could an off-the-shelf mesh codec replace ours? Table 5 runs the same FP32 meshes through Draco [7], the MPEG TFAN coder [17] and zstd, and compares bytes with our file for the same mesh (our encoder with all vertices kept, −0.1 to −0.8 dB). Keeping view dependence as 8-bit SH attributes, Draco needs 12–18× and MPEG 3–4× our bytes; the only generic routes near our size drop the SH altogether (RGB-only vertex colors, or a baked texture) and lose 0.7–4.4 dB. The generic rows are near-lossless and ours is not, so the table measures the bytes each coder needs for essentially the same mesh rather than a rate–distortion curve; The fair generic point is Draco carrying exactly our appearance payload, the two dictionary indices and the 7-bit DC color as one generic attribute on the same positions and faces: its file is 2.7–3.5× ours on every real-scene asset (2.1–3.5× on objects), and Draco's geometry alone, posi-

Table 6. **Ablation.** Size error (mean) and scenes within $\pm 10\%$ of the target; PSNR at the same file size relative to the full method, read on each scene's reference curve (Sec. 4.2). Ficus is capacity-limited at 2.5 MB and excluded.

variant	size error	within $\pm 10\%$	ΔPSNR
Mip-NeRF 360 + T&T, $T = 10$ MB			
full BudgetMesh	$+2.0\%$	10/11	0
no collapse	$+4.5\%$	9/11	-0.03
no byte probe	$+5.0\%$	10/11	$+0.03$
NeRF-Synthetic, $T = 2.5$ MB			
full BudgetMesh	$+2.6\%$	7/7	0
no collapse	$+9.0\%$	2/7	-0.05
no byte probe	$+12.8\%$	1/7	$+0.07$

tions and connectivity with no appearance at all, already weighs 0.85–$1.24\times$ our whole file. Our appearance streams cost 18–42% of our file; coded by Draco's attribute predictor they cost two to three files. The closest route is a hybrid that still depends on our dictionary: Draco for positions and connectivity, zstd for our indices and DC color. It is 1.3–$1.6\times$ our size. So of the gap to generic coders, most comes from replacing the SH coefficients by dictionary indices, a lossy step any coder could adopt, and the last factor of 1.4 from our entropy coding: the implicit vertex order, the edge cache, which matches Draco's connectivity on meshes it was not designed for, and the neighbour contexts for positions and indices. Our meshes are triangle soups stitched late in training and are not manifold, which is what the Edgebreaker-style coders pay for; the mesh statistics are in the supplement.

4.5. Where the bytes go

Splitting a file into its streams (figure in the supplement): connectivity is 39–43% and lossless, positions 20%, appearance indices 17–20%, DC color 11–13%, and the dictionaries a fixed 0.65 MB (3% of a 20 MB file, 12% of a 5 MB one). Two findings follow. *Appearance sharing carries the loss:* 45 of the 52 floats per vertex are SH, replaced by two indices; at the top operating point an 8-bit instead of 7-bit DC color recovers only 0.04–0.08 dB of the 0.4–0.6 dB loss on indoor scenes, so the remainder is the dictionary, and so is the LPIPS increase of Table 3. *Dictionary size should follow vertex count:* over a sweep of dictionary sizes and operating points on all 21 scenes, the fixed $4096 + 256$ is within 0.16 dB of the best choice at every point on captured scenes, but on the two smallest objects (mic, ficus) a 1024-entry dictionary with more vertices is $+0.5$–0.6 dB at the same file size. The sweep is in the supplement; the paper uses the fixed setting.

4.6. Ablation

Table 6 switches off one part of the training rule at a time on all 11 real scenes at 10 MB and all NeRF-Synthetic objects at 2.5 MB, same code, codec and evaluation (ficus is capacity-limited at 2.5 MB and excluded). At 10–20 MB on real scenes the growth plan alone lands within 10% on 9–10 of 11 scenes and none of the parts changes the quality at the same file size by more than 0.03 dB; the collapse rounds and the byte probe each remove one failure mode. The collapse trims the indoor overshoot (kitchen $+20\% \rightarrow +13\%$; on objects it takes the hit rate from 2/7 to 7/7). The probe is what makes small files land: with the fixed constants the 1.5 MB objects land $+18\%$ on average, 1 of 8 within 10%, against $+4\%$ and 8 of 8 with the probe, while at 20 MB the constants are within 10% on 10 of 11. The choice of triangles cannot be switched off as cleanly: with the number of splits per step copied from the full run and the triangles chosen by MeshSplatting's importance score, MeshSplatting's own pruning removes more of the new vertices and the files land 27% (real scenes) and 11% (objects) below the target, so this variant is not a comparison at the same size. The controlled comparisons are Table 1b (same vertex count, $+0.27$ and $+0.24$ dB) and the third row of Table 4 (our probe and plan with MeshSplatting's selection, $+0.07$ to $+0.17$ dB on captured scenes at the same file size).

Qualitative results are in the supplement: decoded files at the three main targets next to the ground truth and the FP32 model, and the same vertex count reached three ways on one view, where the simplified mesh shows holes and speckle and the trained ones do not.

Limitations. The growth aim is a constant per capture class, chosen on the datasets we evaluate on (Deep Blending excepted); it stands in for the faces per vertex of the stitched mesh, which cannot be observed during growth. On captured scenes at 10–20 MB a vertex cap with a good size prior reaches a similar hit rate; BudgetMesh's advantage there is that it needs no prior. The cap-based alternatives of Table 4 were run on four scenes only. Indoor scenes land 5–14% above the target and outdoor scenes at 20 MB up to 7% below. Small objects cannot fill large targets. Trimming (Sec. 3.4) is one-sided: it protects against undershoot from collapsing, not against the growth plan admitting too few splits. Opaque meshes are 3–4 dB below Gaussian codecs at the same file size.

5. Conclusion

Opaque differentiable meshes cannot be shrunk after training, so BudgetMesh decides their size while they grow: it measures the encoded size during training, plans growth toward the vertex count the target can hold, splits where the rendering error is and trims by edge collapse with training still running; a codec with a standalone decoder writes a standard mesh. The file lands within about 10% of the requested size on four datasets, 31–$122\times$ smaller than the trained model, in one training run. The remaining gap to Gaussian codecs is the representation's; closing it is the next step.

References

[1] Jonathan T. Barron et al. Mip-nerf 360: Unbounded anti-aliased neural radiance fields. In *CVPR*, 2022.

[2] Yihang Chen et al. Hac: Hash-grid assisted context for 3d gaussian splatting compression. In *ECCV*, 2024.

[3] Yihang Chen et al. Hac++: Towards 100x compression of 3d gaussian splatting. *arXiv preprint / TPAMI*, 2025. VERIFY.

[4] Zhiqin Chen, Thomas Funkhouser, Peter Hedman, and Andrea Tagliasacchi. Mobilenerf: Exploiting the polygon rasterization pipeline for efficient neural field rendering on mobile architectures. In *CVPR*, 2023.

[5] Zhiwen Fan et al. Lightgaussian: Unbounded 3d gaussian compression with 15x reduction and 200+ fps. In *NeurIPS*, 2024.

[6] Michael Garland and Paul S. Heckbert. Surface simplification using quadric error metrics. In *SIGGRAPH*, 1997.

[7] Google. Draco 3d data compression. https://google.github.io/draco/, 2017.

[8] Antoine Guédon and Vincent Lepetit. Sugar: Surface-aligned gaussian splatting for efficient 3d mesh reconstruction and high-quality mesh rendering. In *CVPR*, 2024.

[9] Peter Hedman et al. Deep blending for free-viewpoint image-based rendering. *ACM Transactions on Graphics (SIGGRAPH Asia)*, 2018.

[10] Hugues Hoppe. Progressive meshes. In *SIGGRAPH*, 1996.

[11] Arseny Kapoulkine. meshoptimizer: Mesh optimization library. https://github.com/zeux/meshoptimizer, 2017.

[12] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. In *ACM Transactions on Graphics (SIGGRAPH)*, 2023.

[13] Arno Knapitsch, Jaesik Park, Qian-Yi Zhou, and Vladlen Koltun. Tanks and temples: Benchmarking large-scale scene reconstruction. *ACM Transactions on Graphics*, 2017.

[14] Joo Chan Lee et al. Compact 3d gaussian representation for radiance field. In *CVPR*, 2024.

[15] Xiangrui Liu et al. Compgs: Efficient 3d scene representation via compressed gaussian splatting. In *ACM Multimedia*, 2024.

[16] Tao Lu et al. Scaffold-gs: Structured 3d gaussians for view-adaptive rendering. In *CVPR*, 2024.

[17] Khaled Mamou, Titus Zaharia, and Françoise Prêteux. Tfan: A low complexity 3d mesh compression algorithm. In *Computer Animation and Virtual Worlds*, 2009.

[18] MeshSplatting authors. Meshsplatting: Differentiable rendering of opaque triangle meshes. In *arXiv / venue TBD*, 2025. VERIFY.

[19] Ben Mildenhall et al. Nerf: Representing scenes as neural radiance fields for view synthesis. In *ECCV*, 2020.

[20] MiLo authors. Milo: Mesh-in-the-loop gaussian splatting for detailed and efficient surface reconstruction. In *SIGGRAPH Asia*, 2025. VERIFY.

[21] Christian Reiser et al. Binary opacity grids: Capturing fine geometric detail for mesh-based view synthesis. In *ACM Transactions on Graphics (SIGGRAPH)*, 2024.

[22] Jarek Rossignac. Edgebreaker: Connectivity compression for triangle meshes. In *IEEE Transactions on Visualization and Computer Graphics*, 1999.

[23] Gary J. Sullivan and Thomas Wiegand. Rate-distortion optimization for video compression. *IEEE Signal Processing Magazine*, 1998.

[24] Jiaxiang Tang et al. Delicate textured mesh recovery from nerf via adaptive surface refinement. In *ICCV*, 2023.

[25] Henan Wang et al. End-to-end rate-distortion optimized 3d gaussian representation. In *ECCV*, 2024.

[26] Yufei Wang et al. Contextgs: Compact 3d gaussian splatting with anchor level context model. In *NeurIPS*, 2024.

[27] Lior Yariv et al. Bakedsdf: Meshing neural sdfs for real-time view synthesis. In *SIGGRAPH*, 2023.

[28] Zehao Yu, Torsten Sattler, and Andreas Geiger. Gaussian opacity fields: Efficient adaptive surface reconstruction in unbounded scenes. In *ACM Transactions on Graphics (SIGGRAPH Asia)*, 2024.

[29] Baowen Zhang et al. Rade-gs: Rasterizing depth in gaussian splatting. *arXiv preprint*, 2024.